

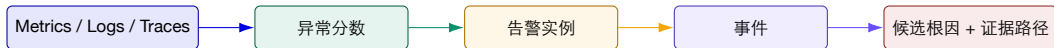
面向 AI 原生系统的智能运维

第 7 讲 | 多维根因定位与诊断证据组织

调用链、HotSpot、规则挖掘、多模态事件图与实验 2

陈鹏飞 | 中山大学

从第 6 讲继续：异常分数还不是根因



第 6 讲 判断“何时偏离正常”。

第 7 讲 判断“谁先变坏、如何传播”。

第 8 讲 讨论“什么证据支持因果”。

增量归因算例：异常最多的区域就是根因吗？

两个等长窗口内的错误请求计数如下；先计算增量贡献，再决定检查方向。

区域	基线错误数	事故错误数	增量 / 总增量
东区	20	80	$60 / 100 = 60\%$
南区	30	60	$30 / 100 = 30\%$
西区	10	20	$10 / 100 = 10\%$
合计	60	160	增加 100

优先检查东区能覆盖最大的错误增量；下一步需要请求总量、版本和调用路径。

推导与判断 贡献率只说明“新增错误在哪里”。如果东区流量也增长到原来的 4 倍，错误率可能没变；未除以请求总量之前，不能将错误计数当作风险。

算例使用假设数据。

开场事故：缓存热键如何在 30 秒内变成全链路故障？

脱敏案例时间线

- ▶ 购物车服务开始出现尾延迟；
- ▶ Redis 热键过期，回源流量打满 DB 连接池；
- ▶ 重试与超时沿调用链放大；
- ▶ 12 个服务在约 30 秒内出现症状。

为什么“最高告警”不是根因？

- ▶ DB pool full 是中间状态；
- ▶ 下游 timeout 是传播结果；
- ▶ 最早变化可能藏在缓存配置变更；
- ▶ 需要时间、拓扑、trace、日志和 change 一起验证。

定位原则 根因定位是“竞争性假设的证据更新”，不是按告警数量排序。

把 RCA 写成可证伪的问题

时间问题 哪个对象最早偏离？异常先于还是晚于变更？

空间问题 服务、操作、实例、资源中哪一层先出现证据？

传播问题 沿调用链、共享资源还是发布批次传播？

语义问题 日志、错误码、配置差异是否支持解释？

首发证据

传播路径

根因候选

反证与动作

本讲路线图：从调用链到多模态证据

1. RCA 输入输出要求：症状、候选、根因、证据和反证；
2. 调用链：trace coverage、PageRank 和 spectrum ranking；
3. 多维归因：HotSpot 风格分解、决策树和规则挖掘；
4. 多模态：Nezha 事件图和 Expected/Actual Pattern；
5. 变更：ChangeRCA、canary、DiD 与三类 scorer；
6. AI 系统：TELLER 的跨层 request-level call-chain；
7. 实验 2：时序检测、候选排名和多模态消融。

诊断要求：候选必须带上下文

字段	必须回答	不完整的说法
对象	服务/实例/操作/资源/租户	“DB 异常”
时间	首次偏离、窗口、持续时间、时区	“刚刚变慢”
路径	上游、下游、资源、变更边	“模型分数最高”
证据	metric/log/trace/topology/change 的具体记录	“系统判断”
反证	哪些观察削弱该候选	“没有发现其他原因”
动作	回放、隔离、回滚、扩容或继续观察	“建议关注”

判断条件 不能回答“相对谁、何时发生、如何验证”，就只能称为可疑信号。

一、从症状到证据

根因分析的第一步不是选模型，而是定义候选空间和证据关系。

异常、故障、告警和事件不是同一层



- ▶ 促销流量可能是统计异常，但不是故障；
- ▶ 静默数据损坏可能是故障，却未触发明显指标；
- ▶ 一个故障可触发上千条告警；
- ▶ 一个事件需要在多个候选之间保留不确定性。

四类假设解释同一次告警风暴

H1 上游传播 上游节点先异常，下游沿依赖滞后恶化。

H2 局部资源 单个操作、实例或 GPU/连接池出现异常。

H3 变更诱发 部署或配置变更改变了处理组 KPI。

H4 观测伪影 采样、时钟、ID 或规则自身制造了证据。

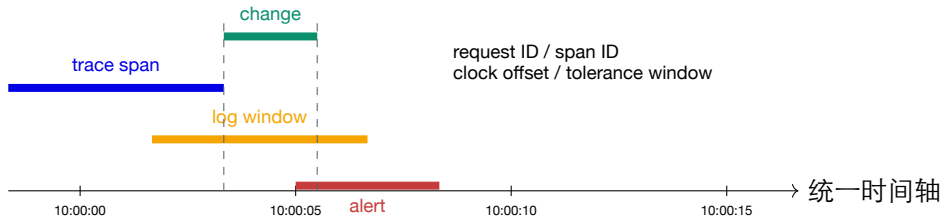
验证顺序 先检查观测与变更，再看用户影响和首发对象，最后讨论模型调参。

多源数据的角色分工

数据源	最强证据	能回答什么	常见盲点
指标 日志 Trace 拓扑 Change	数值偏离、持续性、同伴差异 错误码、模板、异常语义 parent-child、耗时、调用路径 依赖、共享资源、变更范围 发布、配置、流量、模型或硬件 操作	何时变坏、影响多大、谁异常 发生了什么类型的失败 谁调用谁、哪里变慢、如何传播 候选传播边与影响面 哪个干预可能改变了状态	语义弱，容易只看到症状 稀疏、重复、缺 request ID 高基数、采样、时钟问题 静态图可能过时 变更与事故时间可能重叠

一个源提供“强线索”，多源对齐才可能提供“可审计证据链”。

数据对齐是 RCA 的地基



危险 时间相近不是因果；但没有统一时间与 ID，连“先后”都无法可靠讨论。

候选根因的最小数据结构

Candidate

node, score, first_seen, path, evidence[]
supporting[], contradicting[], action

Evidence

source, timestamp, object, relation, value,
confidence

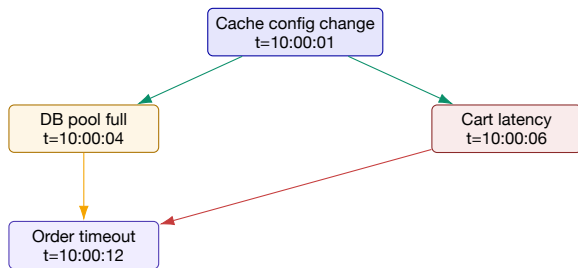
Candidate 还没有被确认的解释。

Root cause 经过对照或实验验证后，最能解释首发与传播的候选。

Evidence path 从症状回溯到候选的可复查边序列。

实验要求每个 top-k 候选至少有一条支持证据和一条可执行的下一步动作。

为什么只按异常分数排序会错？



分数最高：order timeout
首发证据：cache config change
合理排序：按时间 + 路径 + 影响

排序原则 异常强度只是一个维度；时间先后、路径连接、变更关系和负证据共同决定候选优先级。

证据矩阵：把“支持/反对”显式写出来

候选	指标	日志	Trace	拓扑	Change	解释
Cache change	+	0	+	+	++	早于 DB pool，且影响同一入口
DB pool bug	++	+	+	+	0	能解释连接耗尽，但不能解释首发变更
网络抖动	+	0	+	+	0	需要跨区域/跨服务的时序证据
检测器漂移	+	0	0	0	0	若原始 SLO 未变化则优先检查观测链

“0” 不是证明不存在，而是当前窗口没有支持证据；证据状态需要随新数据更新。

首发、传播、恢复：三段时间比一张截图更有用

首发

- ▶ 第一个异常节点
- ▶ 首次错误日志
- ▶ 变更前后差异

传播

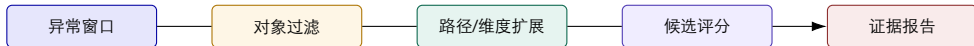
- ▶ trace 路径
- ▶ 共享资源
- ▶ 受影响服务集合

恢复

- ▶ 回滚/隔离后谁先恢复
- ▶ 告警是否仍在
- ▶ 反事实线索

注意事项 恢复顺序可以削弱某些候选，但“回滚后恢复”仍需考虑同时发生的流量或缓存变化。

从第 6 讲的异常区间进入第 7 讲的候选集合



- ▶ 异常检测先减少时间和对象搜索空间；
- ▶ RCA 再引入调用关系、属性组合、事件模板和变更；
- ▶ “候选生成”和“候选确认”要分开，避免早期过度剪枝。

二、调用链分析：从“谁异常”到“谁可能导致异常”

Trace 给出结构和时序，但必须从原始 span 组织为可排序的候选图。

Trace 的三种视角

时间线 每个 span 的开始、结束和 self-time。

覆盖树 同一请求中 parent-child 的嵌套关系。

操作图 跨多个请求聚合后的服务/操作调用关系。

span

parent

child

operation

candidate

MicroRank: 为什么不能把所有 trace 当成同一种测试用例?

- ▶ 不同请求覆盖的操作集合不同;
- ▶ 处理时间与调用深度不同;
- ▶ 只看端到端延迟会把复杂但正常的请求误判为异常;
- ▶ 需要比较“异常 trace 与正常 trace 的结构差异”。

输入 trace latency、operation handling time、coverage graph、call graph。

输出 服务操作级 root-cause ranking, 而不是一句泛化告警。

MicroRank 的 trace coverage tree

Latency Issue Localization with Extended Spectrum Analysis in Microservice

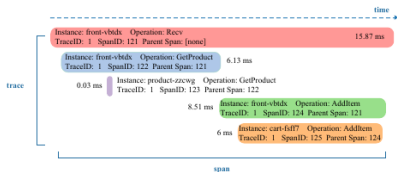


Figure 1: An example of trace in Hipster-Shop

WWW '21, April 19–23, 2021, Ljubljana, Slov

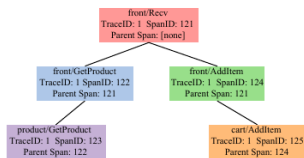


Figure 2: The trace coverage tree of Fig. 1

左侧是一次 Hipster-Shop trace，右侧把 span 的 parent-child 关系折叠为 coverage tree；多条 trace 聚合后得到 operation call graph。

第一步：识别异常 **trace**，而不是只识别异常指标

论文中的可解释基线

对每个操作估计正常处理时间 μ_o, σ_o ，按 **trace** 覆盖次数计算期望延迟：

$$L_{expected} = \sum_o count_o(\mu_o + n\sigma_o).$$

若实际端到端延迟超过期望，则该 **trace** 进入异常集合。

- ▶ 论文采用短滑窗，触发后避免重复定位；
- ▶ n 控制上界松紧；
- ▶ 关键是利用 **trace** 结构，而非固定长度向量；
- ▶ 该基线可替换为第 6 讲的任意检测器。

第二步：把 trace 变成 operation-trace graph



Figure 6: The anomalous operation-trace graph of Fig. 3

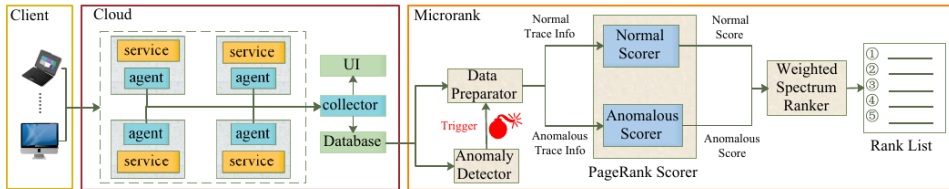
ally determines whether the operations occur but ignores the re
onships among operations. Only when the root cause localizati
hase is triggered, the *Data Preparator* module in *MicroRank* w
extract detailed trace information about the operations' relat

- ▶ 节点：服务实例操作；边：操作之间的调用关系；
- ▶ trace 作为证据集合，连接“哪些操作共同出现”；
- ▶ 正常图和异常图分开构造，为后面的相对评分提供输入。

MicroRank 框架：异常检测、数据准备、双 PageRank、频谱排序

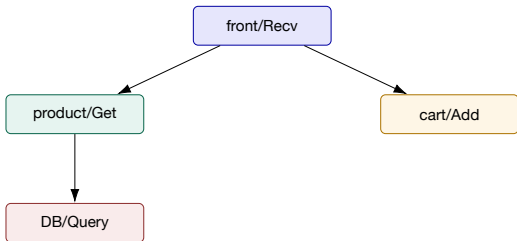
Agency Issue Localization with Extended Spectrum Analysis in Microservice

W. W. W. et al., April 12-23, 2021, Guojiang, China



读图顺序：先看输入图，再看 Normal/Anomalous Scorer，最后看 Weighted Spectrum Ranker 如何合并相对证据。

PageRank 直觉：重要性来自“被哪些 trace 支持”



- ▶ 一个节点被多条异常 trace 支持，分数上升；
- ▶ 若同样被大量正常 trace 支持，异常解释力下降；
- ▶ 图结构让“共同出现”比单点分数更有上下文；
- ▶ 不是把 PageRank 当作因果概率。

警示：图上的高分表示“更值得检查”，不等价于已证实的缺陷。

Weighted Spectrum: 把异常/正常差异转成排名

计算表示

对候选操作 o ，可将异常支持、正常支持和图结构合成为：

$$\text{Score}(o) = w_a A(o) - w_n N(o) + w_g G(o).$$

其中 $A(o)$ 是异常图支持， $N(o)$ 是正常图支持， $G(o)$ 表示结构相关性；实际实现由论文的 PageRank 与 spectrum ranking 共同决定。

异常高 更常出现在异常 trace。

正常高 也常出现在正常 trace，需降权。

结构高 位于共同传播路径或关键操作。

一个手算例子：为什么中间服务可能被降权？

操作	异常 trace 支持 A	正常 trace 支持 N	路径相关 G	直觉分数	解释
Cache/Get	8	1	0.9	高	首发附近且很少出现在正常窗口
DB/Query	10	7	0.8	中	既是症状节点，也经常处理正常流量
Order/Commit	9	2	0.6	中高	下游影响大，但时间上较晚

读表方法 最终排序需要同时看分数、首次出现时间和路径；不能因为 DB 的告警更多就把它当作根因。

论文报告的 **MicroRank** 结果如何正确解读？

- ▶ 论文报告相对既有方法约 6–22% 的 recall 提升；
- ▶ 单故障数据集上 R@1 报告为 88% 以上；
- ▶ 双故障数据集上 R@3 超过 60%，可定位其中一个故障超过 90%。
- ▶ 这些是论文 benchmark/故障注入设置下的结果；
- ▶ R@k 只说明真根因是否进入前 k；
- ▶ 生产还要看 EXAM、调查时间、采样覆盖和解释可读性；
- ▶ 实验应复现“排名逻辑”，不承诺复制论文数字。

MicroRank 的边界：图是输入，不是自动因果图

1. trace 采样遗漏关键边时，候选集合本身已经不完整；
2. clock skew 或异步调用会破坏简单的时间排序；
3. 共享资源故障可能没有稳定的 service operation 节点；
4. 多故障同时发生时，排名会把路径混合在一起；
5. 需要把 trace 排名与指标、日志、拓扑和 change 交叉验证。

这正是 Nezha 和 ChangeRCA 引入多模态/变更证据的动机。

三、HotSpot 与规则挖掘

当 KPI 是多个维度的加和，先解释“哪一类贡献了异常”，再向服务/实例钻取。

HotSpot 风格问题：总量异常来自哪一个维度？

可加 KPI

例如每分钟错误请求数：

$$KPI(t) = \sum_{region} \sum_{service} \sum_{status} count(region, service, status, t).$$

当总量上升时，问题不是“总量是否异常”，而是“哪个属性切片贡献了增量”。

切片 region= 广州、
service=cart。

贡献 该切片占总增量
63%。

钻取 继续看实例、版本、
接口和状态码。

维度贡献的计算直觉

贡献率计算

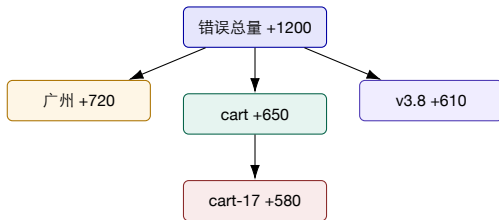
对等长窗口 B, F , KPI 取错误数等可加总计数; g 为同一维度下互斥且完备的切片:

$$\Delta_g = KPI_g(F) - KPI_g(B).$$

$$Contribution(g) = \frac{\max(\Delta_g, 0)}{\sum_h \max(\Delta_h, 0) + \epsilon}.$$

- ▶ 错误率与 P95 不能按此式直接相加; 先用分子、分母或桶计数;
- ▶ 对负贡献、基线漂移和稀疏小样本要单独处理;
- ▶ 贡献率解释 “哪里值得看”, 不证明 “谁导致了它”。

HotSpot drill-down: 从总量到可操作对象



- ▶ 每一层都要保留分母、基线窗和样本量；
- ▶ 维度之间可能重叠，不能简单把贡献相加；
- ▶ 最后一层才连接 **trace**、日志和变更，形成候选路径。

维度归因的三个陷阱

辛普森悖论 总体上升不代表每个 region 都上升；聚合顺序会改变结论。

小样本放大 一个实例的 2 个错误可能占比 100%，但不代表影响最大。

维度泄漏 service、route、版本字段可能描述同一个事件，重复计数。

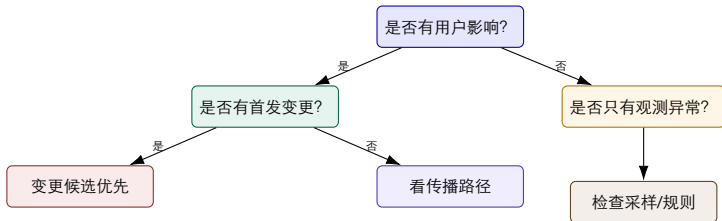
工程要求 每个贡献必须给出样本数、基线、置信区间或最小支持度。

一个电商例子：贡献排序如何改变调查顺序？

切片	故障窗错误数	基线窗错误数	增量	调查动作
广州 / cart / v3.8	910	300	+610	查最近发布与缓存配置
深圳 / order / v3.7	420	250	+170	作为传播结果核对 trace
广州 / search / v3.8	380	360	+20	低优先级，先排除背景波动

诊断问题 如果 trace 显示 order 先于 cart 异常，应该如何更新这张表的调查顺序？

决策树：把专家调查顺序写成可复查规则



决策树的价值是显式化判断条件；它不能替代 trace 或日志，只负责排序和剪枝。

规则挖掘：从共现到候选假设

规则模板（示例）

```
IF error_rate(cart)>x AND db_pool_wait>y AND cache_miss_rate>z  
AND first_seen(cache_config_change) < first_seen(db_pool_wait)  
THEN rank(cache_config_change) high
```

支持度 规则在多少故障窗口成立？

置信度 满足前件时，候选多常进入 top-k？

反例 哪些窗口满足前件但候选并非根因？

关联规则不是因果证明

可以支持的结论

- ▶ 两类事件在同一窗口经常共现；
- ▶ 某个条件组合适合用于候选筛选；
- ▶ 规则可转成可审计的调查顺序。

还不能支持的结论

- ▶ 前件必然导致后件；
- ▶ 删除前件就一定恢复；
- ▶ 在新业务/新版本中仍保持同样效果。

桥接第 8 讲 需要干预、反事实或结构因果模型，才有机会从关联走向因果。

四、Nezha：把多模态数据变成事件图

当指标、日志和 trace 有不同粒度时，先统一事件表示，再谈模式和排名。

Nezha 的问题定义：服务级 RCA 太粗

- ▶ 指标告诉我们“系统状态变坏”；
- ▶ 日志告诉我们“出现了什么语义”；
- ▶ trace 告诉我们“请求如何经过服务”；
- ▶ 只在服务级输出会隐藏真正可修复的操作/资源。

Nezha 的主张 将异构观测转换为带时间、trace/span ID 的统一事件，再用构造期与故障期的模式差异生成细粒度候选。

Nezha 总览：从观测数据到排名列表

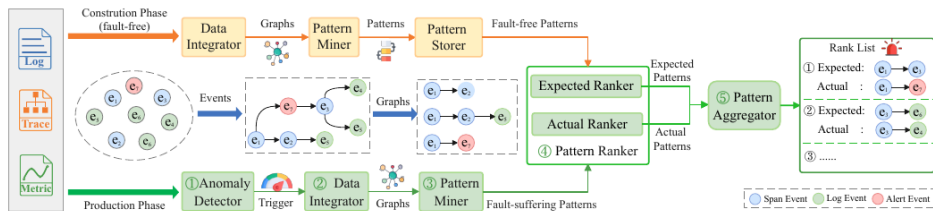


Figure 4: The overview of Nezha. Nezha uses the multi-modal observability data as input and outputs the ranked list of

上游 construction phase 学习 fault-free patterns；生产窗口经 anomaly detector 触发，再构造 fault-suffering patterns。

统一事件表示：每一种模态都回答“此刻发生了什么”

事件示意

event = (type, service, operation,
trace_id, span_id, ts, value)

- ▶ Metric：只把可疑指标告警转成事件；
- ▶ Log：模板 + 动态参数 + 时间/ID；
- ▶ Trace：span start/end 与 parent-child；
- ▶ Alert：在窗口内反复出现的告警也可成为事件。

为什么先过滤？ 将所有正常指标点都塞进图会放大噪声；Nezha 只使用有诊断价值的异常指标事件。

关键代价 事件化降低异构性，但会丢掉正常背景和细粒度原始数值。

从原始 trace 到 trace events

1 observability data as input and outputs the ranked list of

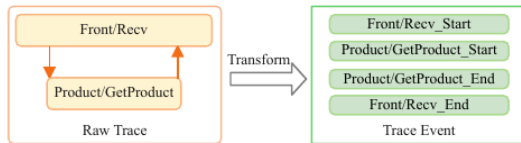
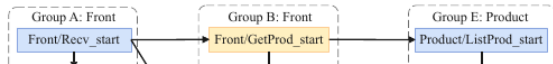


Figure 5: Transformation from a raw trace to trace events.



- ▶ 同一 span 拆成 start/end 两个事件，保留持续时间；
- ▶ parent-child 与时间戳共同决定事件顺序；
- ▶ 统一事件格式为后续 pattern miner 提供输入。

Event graph: 异步调用也需要结构边

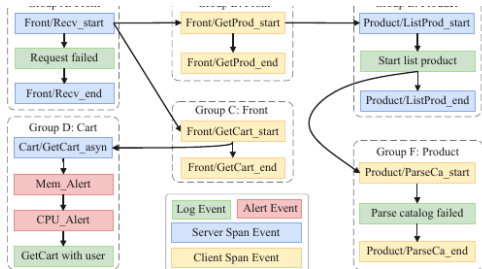


Figure 6: An example of event graph in OnlineBoutique. All trace events and log events have the same trace ID.

节点 Span、log、metric、alert event。

边 调用、时序、同trace、资源或服务关系。

目的 让模式可排序、可解释、可追溯。

Expected Pattern 与 Actual Pattern

Expected Pattern Ranker

- ▶ 在 fault-free construction phase 中频繁；
- ▶ 在 fault-suffering window 中很少；
- ▶ 代表“本应发生但缺失”的执行模式。

Actual Pattern Ranker

- ▶ 在 fault-suffering window 中频繁；
- ▶ 在 fault-free phase 中很少；
- ▶ 代表“故障时新增或替代”的执行模式。

解释方式 把“缺失的正常模式”和“新增的故障模式”配对，比只找异常节点更容易形成可读报告。

Pattern Aggregator: 把两条排名变成一条候选链

angba Yu, Pengfei Chen, Yufeng Li, Hongyang Chen, Xiaoyun Li, and Zibin Zheng



Figure 7: Examples of event graphs in construction and production phase. Req_i denotes the i th kind of request (e.g., login or query product). The pattern $e_2 \rightarrow e_3$ in G_C turned into $e_2 \rightarrow e_6$ in G_P when a fault occurred.

Table 2: Example of *Nezha* to perform RCA from Fig. 7.

Pattern	Support		Score		Depth	Rank($Score_E$)
	S_C	S_P	$Score_E$	$Score_A$		
$e_1 \rightarrow e_2$	3	3	0.5	0.5	1	2

- ▶ Expected: $e_1 \rightarrow e_3$ 在正常请求中稳定出现；
- ▶ Actual: 故障窗口出现 $e_1 \rightarrow e_6$ ；
- ▶ 配对后的结果包含“正常路径被什么替代”的语义。

Nezha 的排名列表如何读？

servability Data ESEC/FSE '23, December 3–9, 2023, San Francisco, CA, USA

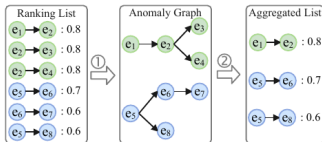


Figure 8: Example of constructing anomaly graphs to get an aggregated list in Pattern Aggregator.



1. 先看候选事件对和其发生/缺失关系；
2. 再看 trace ID、span ID 和服务操作；
3. 最后把候选映射回指标、日志和变更证据。

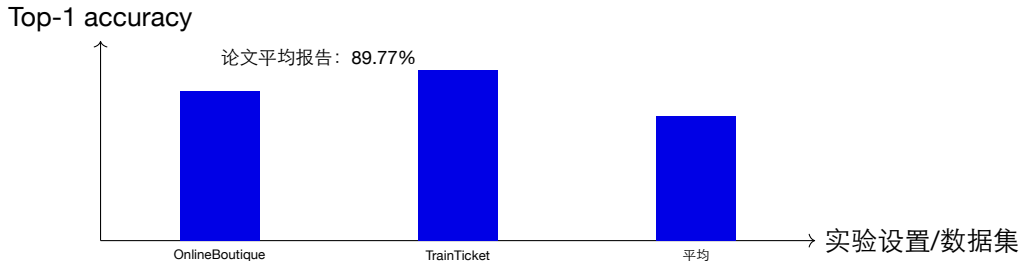
Nezha 的异常检测入口：简单但必须可审计

论文设定

对 success ratio 与 P90 latency 使用 k -sigma 规则，默认 $k = 3$ ；进入生产窗口后，超过阈值才触发 RCA。

- ▶ 这不是 Nezha 的唯一贡献，而是可替换入口；
- ▶ 重点是“何时构造故障事件图”；
- ▶ 应记录指标层级，避免把应用层症状当作根因证据；
- ▶ 规则阈值必须按业务和季节校准。

论文结果：89.77% top-1 该如何表述？



- ▶ 这是作者在论文数据与根因粒度下的报告值；
- ▶ 不应写成“Nezha 在所有生产系统达到 89.77%”；
- ▶ 复现实验应报告自己的数据切分、根因粒度和 top-k 定义。

多模态融合的真实成本：关联质量决定上限

时钟 不同机器的时间基准偏移。

标识 日志没有 request/span ID。

覆盖 采样遗漏关键操作或资源。



如果 align 这一层不可信，后面的图模型只会把错误关联放大。

Nezha 小结：事件化带来解释，但不消除不确定性

1. 指标、日志、trace 先统一为事件；
2. parent-child、时间和 ID 共同构造 event graph；
3. Expected/Actual pattern 形成“正常被什么替代”的解释；
4. 排名输出仍需人工验证变更、拓扑和恢复证据；
5. 下一步问题：如何把软件变更本身纳入 RCA？

五、ChangeRCA：把变更作为独立证据

很多事故不是“某服务异常”，而是“某次变化改变了系统状态”。

RCA 与 RCCA 的对象不同

传统 RCA

- ▶ 排名服务、实例、操作或资源；
- ▶ 关注异常传播路径；
- ▶ 输出“哪里可能坏了”。

Root Cause Change Analysis

- ▶ 排名部署、配置、流量、模型或硬件变化；
- ▶ 区分 canary change 与 non-change fault；
- ▶ 输出“哪次变化值得优先回滚/检查”。

关键差异 同一个服务可同时有多个变化；“服务 L 异常”并不告诉我们该检查哪次变更。

ACD 与 RCCA: 为什么只看 KPI 差异还不够?

2:4

Guangba Yu, Pengfei Chen, Zilong He, Qiuyu Yan, Yu Luo, Fangyuan Li, and Zibin Zheng

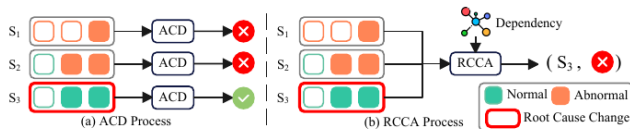
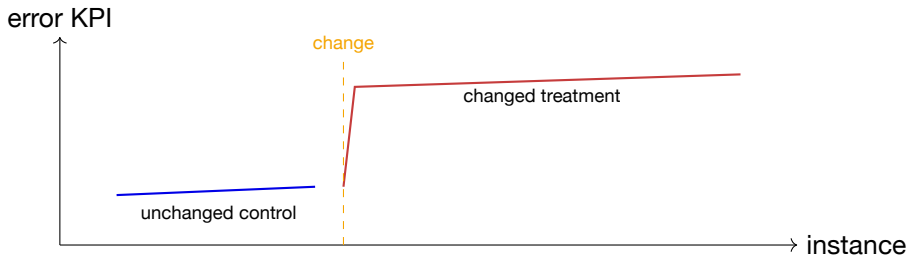


Fig. 2. Comparison between abnormal change detection (ACD) and root cause change analysis (RCCA).

ACD 独立处理每个变化；RCCA 将异常服务、依赖关系和变化流合并，允许候选变化来自传播链。

Canary 对照：区分组别与发布前后窗口



- ▶ 在同一发布后窗口，若只有处理组恶化，变化归因更有支持；
- ▶ 若处理组与对照组同步恶化，应考虑共同外因；
- ▶ 仍需排除同时发生的流量、依赖或基础设施变化。

Difference-in-Differences 的直觉

两层差分

$$DiD = (Y_{post,treat} - Y_{pre,treat}) - (Y_{post,control} - Y_{pre,control}).$$

若处理组相对对照组的变化显著更大，变更诱发的解释得到支持。

前提 对照组能代表未受变更影响的趋势。

窗口 前后窗要避开冷启动和其他发布。

边界 DiD 是证据，不是自动因果证明。

Change flow: 根变化可能没有直接 KPI 异常

2:12 Guangba Yu, Pengfei Chen, Zilong He, Qiuyu Yan, Yu Luo, Fangyuan Li, and Zibin Zheng

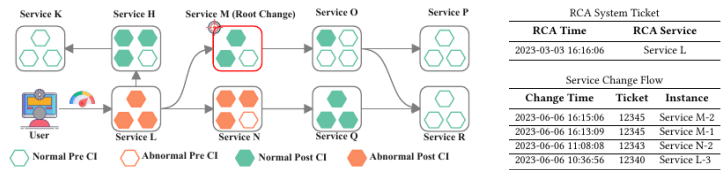


Fig. 8. Part of dependency graph and change flow of the case in Fig 5(b). Service H, L, M, N, O and Q undergo software changes. Service M encounters a defective change and propagates faults to L and M.

论文案例中，Service M 的 root change 通过依赖传播到 L/N 等服务；根变化自身可能没有最明显的 KPI。

ChangeRCA 框架：三阶段缩小变化空间

ChangeRCA: Finding Root Causes from Software Changes in Large Online Systems

2:9

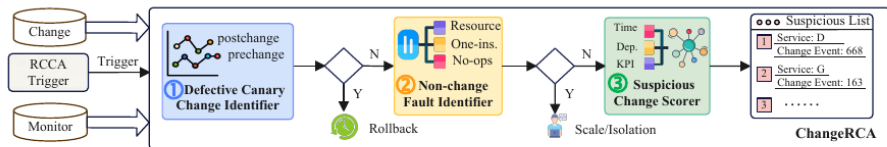


Fig. 6 ChangeRCA framework

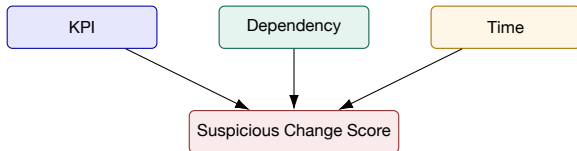
1. Defective Canary Change Identifier: 检查变化前后差异;
2. Non-change Fault Identifier: 过滤明显不是变化导致的故障;
3. Suspicious Change Scorer: 融合 KPI、依赖和时间证据。

三个 scorer: 同一候选的不同证据面

KPI Scorer 变化后异常
实例/指标增量是否更大?

Dependency Scorer 候
选变化是否位于异常服务
的依赖路径?

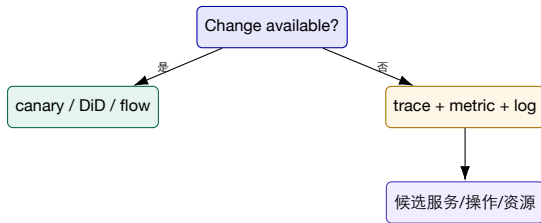
Time Scorer 变化时间
与故障时间是否吻合?



ChangeRCA 结果与正确表述

- ▶ 论文在 WeChat 生产数据与 OnlineBoutique benchmark 上评估；
- ▶ 报告 HR@1 约 85%，HR@3 约 96%；
- ▶ 论文还讨论了调查时间（TTI）下降。
- ▶ HR@k 表示真实变化是否进入前 k；
- ▶ 结果依赖 change log 完整性与 RCA trigger；
- ▶ 没有可靠变更记录时，系统只能退回服务级候选；
- ▶ 不能把论文数字写成企业普适 SLA。

没有变更事件时怎么办？

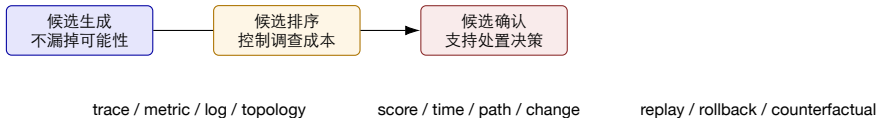


工程建议 将“无变更记录”本身记录为观测缺口；不要让模型假装看到了不存在的干预。

六、证据组织与不确定性

候选排名之后，必须把支持证据、反证和下一步验证动作一起交付。

候选生成、候选排序、候选确认是三件事



过早确认会漏根因，过晚排序会增加调查时间；三阶段应有不同评价指标。

候选评分不应隐藏证据权重

课程示意

$$S(c) = w_t T(c) + w_p P(c) + w_m M(c) + w_l L(c) + w_c C(c) - w_n N(c).$$

- ▶ T : 首发与时间先后; P : 调用/拓扑路径;
- ▶ M : 指标或维度贡献; L : 日志语义; C : 变更关联;
- ▶ N : 负证据, 例如候选在大量正常窗口中也出现。

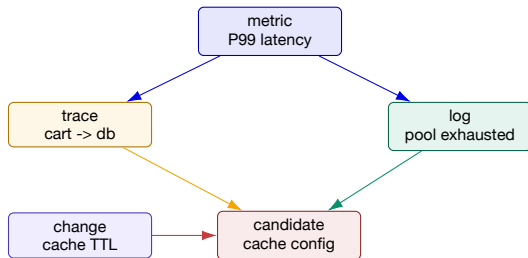
可解释性要求 页面上展示分量和来源, 而不是只展示一个无法审计的 0.87。

负证据：为什么“没有看到”也要记录？

负证据	对候选的削弱	可能的替代解释
候选节点在故障前已持续异常 日志错误模板只出现在下游 trace 中缺少候选 parent 回滚后候选仍异常	不像本窗口的首发变化 更像传播症状 路径证据不完整 回滚解释被削弱	长期容量问题、慢性泄漏 上游资源、网络或配置 采样遗漏、异步队列、跨进程关联失败 共享依赖、流量或观测链路

负证据不是“证明候选错误”，而是调整下一步验证动作的优先级。

把多源证据画成一条可复查路径



报告格式 首发时间 + 传播路径 + 语义日志 + 变更事件 + 反证 + 下一步动作。

数据缺口也要进入诊断报告

缺 **ID** 日志无法对齐请求；
降低日志证据置信度。

缺 **parent** 只能做时间邻近，不应宣称调用路径。

缺 **change** 不能使用
DiD；转为服务级排序并标记缺口。

evidence coverage

alignment confidence

uncertainty

什么时候应停止自动排序，交给人工？

1. top-1 与 top-2 分数接近，且证据来自同一模态；
2. 数据覆盖低于诊断要求，或时钟/ID 无法对齐；
3. 发生多个同时故障，候选路径互相交叉；
4. 自动动作可能造成高风险回滚、扩容或数据变更；
5. 模型遇到训练分布外的服务、版本或资源类型。

安全原则 自动化应该缩小搜索空间和组织证据，不应在证据不足时伪造确定性。

七、LLM 推理系统的调用链

AI 系统的根因定位需要把“模型服务语义”连接到底层执行与通信事件。

LLM 推理链：一次请求跨越多少层？



- ▶ TTFT 变慢可能来自 queue、prefill、KV cache 或 GPU;
- ▶ decode 吞吐下降可能来自通信、kernel 或 batch 调度;
- ▶ 服务日志只看到“请求超时”时，仍需下钻跨层 trace。

TELLER: 跨层 trace + 日志的 request-level RCA

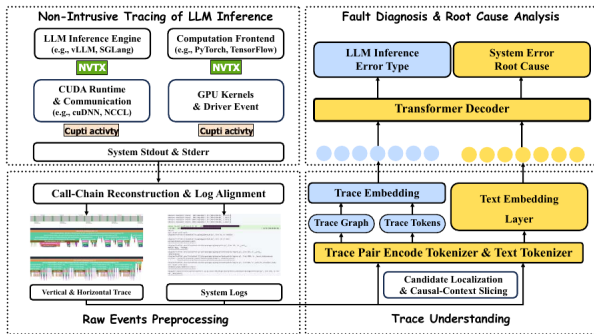


Figure 1: Overview of TELLER. TELLER first performs non-intrusive cross-layer tracing of LLM inference, then reconstructs

TELLER 的确定性前半段负责采集、重建和对齐；后半段再做候选定位、结构压缩和多模态解释。

TELLER 的非侵入式观测思路

- ▶ NVTX: engine 与 framework 层语义范围;
- ▶ CUPTI: runtime、driver、GPU kernel 和 correlation ID;
- ▶ stdout/stderr: 与同一执行关联的文本证据;
- ▶ 不修改模型 binary, 降低接入侵入性。

重建结果 从 raw events 得到 per-request call-chain tree, 覆盖 engine、frontend、backend、host、device、communication。

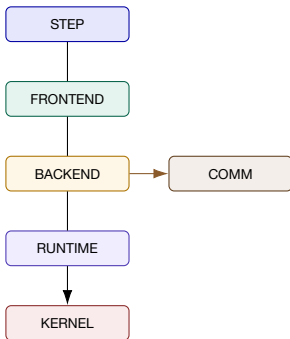
关键判断 非侵入采集解决“拿到证据”, 不自动解决“解释证据”。

请求级 call-chain 的三个结构关系

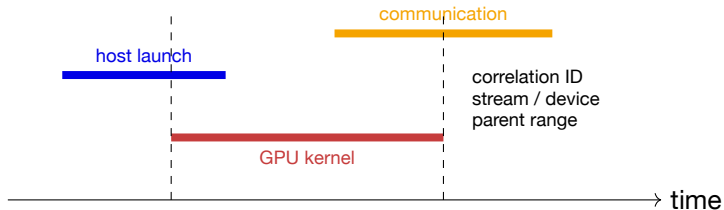
垂直关系 engine step ->
frontend -> backend ->
host -> device。

水平关系 跨节点通信、
collective、同步和传输。

时间关系 同一步骤内的
相邻事件与日志窗口。



为什么“时间邻近”在异步 GPU 系统中不够？



判定 需要 thread、correlation、parent、stream 等结构 ID；仅凭相邻 timestamp 可能把不同请求或不同 kernel 错配。

传统 RCA 与 TELLER 的桥接关系

传统微服务 RCA	LLM 推理对应物	可迁移的方法
service operation	engine step / frontend op / kernel	候选节点要有层级与粒度
trace coverage tree	vertical execution tree	parent-child 保留结构解释
call graph	cross-node communication graph	水平边描述传播/耦合
log template	runtime / scheduler / CUDA error	语义证据与时间窗口对齐
change event	model/config/driver/deployment event	变化是独立证据源

TELLER 不是把 LLM 当作普通微服务，而是把多层执行语义纳入同一 RCA 输入输出要求。

八、实验2：从异常检测到候选排名

先实现可解释基线，再逐步加入 trace、日志、拓扑和变更。

实验 2 的数据格式约定

字段组	样例字段	用途
Trace	trace_id, span_id, parent_span_id, service, operation	调用链与路径
Time	ts_start, ts_end, window_id, timezone	首发、持续、延迟
Metric	name, value, labels, unit	异常分数与维度贡献
Log	template, level, message, request_id	语义证据与错误类型
Topology	caller, callee, resource, version	传播候选与共享依赖
Change	change_id, type, start, target, canary	变更关联与 DiD
Label	root_cause, fault_window	训练/评测，禁止提前泄漏

实验约束 所有时间窗、阈值、模式库和规则必须只读取过去或训练窗口。

实验 2：必做异常检测，选做多模态候选排序

基础方案

- ▶ 必做：真实采集且带异常区间标注的时序数据；
- ▶ 选做：有对齐 Trace、日志、拓扑和根因标签的基准；
- ▶ Online Boutique / TrainTicket 是应用，需配套采集与标注。

企业替换接口

- ▶ 脱敏后映射到同一 schema；
- ▶ 保留时间、对象、依赖和变更关系；
- ▶ 报告真实标签获取方式与覆盖率。

只有时序标签时评测检测效果；没有根因标签和多模态数据，不要求计算 HR@k 或完成 Baseline C/D。

Baseline A: 稳健时序异常检测

仅使用过去窗口

$$\begin{aligned} baseline_t &= median(x_{t-w:t-1}), & scale_t &= MAD(x_{t-w:t-1}) + \epsilon, \\ score_t &= 0.6745 \frac{|x_t - baseline_t|}{scale_t}. \end{aligned}$$

- ▶ 先检测异常区间，再按持续性合并；
- ▶ 低流量、缺失、季节性和发布窗口要单独标记；
- ▶ 记录窗口、阈值、随机种子和版本，便于回放。

Baseline B: Isolation Forest 只解决“孤立”，不解决路径

- ▶ 特征：延迟、错误率、队列、连接池、请求量、同伴差异；
- ▶ 随机切分树，路径短的样本更可能是孤立点；
- ▶ 需要滚动特征或时间窗，不能把未来值拼进当前样本。

RCA 缺口 Isolation Forest 可生成对象候选，却不知道调用路径、日志语义或变更关系。

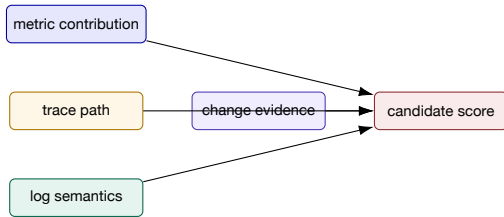
实验比较 比较“IForest top-k”与“IForest + trace/log/change 证据”的 HR@k 差异。

Baseline C: trace candidate ranking



1. 用异常检测器生成 trace 窗口；
2. 按 parent-child 聚合 operation coverage；
3. 比较异常/正常支持，输出 candidate、score、path。

Baseline D: 多模态候选排名

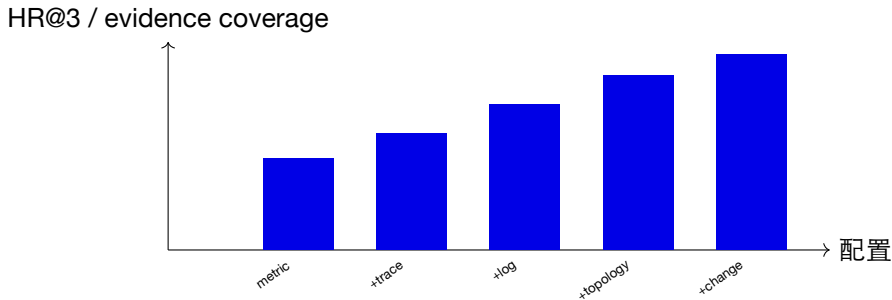


实验重点不是训练大模型，而是验证：加入一种证据后，候选排名、解释覆盖和调查成本如何变化。

评测指标：不要只报告一个 F1

指标	解释	本实验问题
Point F1	点级异常判断	是否容易被区间/标签定义影响？
Range/Event Recall	异常区间或事件是否被覆盖	是否发现了一整次事故？
HR@k	根因是否进入前 k	值班人员需要检查几项？
EXAM/调查成本	排名中需要检查多少候选	top-1 失败时是否仍能节省时间？
Detection delay	首次检测距离真实开始多久	早发现是否以更多误报换来？
Evidence coverage	候选有多少支持/反证字段	结果能否被复查和处置？

实验消融：每个模态到底贡献了什么？



- ▶ 这是实验设计示意，不预设哪种模态一定提升；
- ▶ 如果加入模态后 HR@k 上升但解释覆盖下降，应检查关联错误；
- ▶ 同时报告缺失率和运行开销，避免只挑最好指标。

谁是值得检查的候选？

候选	首发顺序	trace 路径	日志	change	你的排序理由
A: cache config	1	上游入口	miss spike	10:00:01	
B: DB pool	2	中间节点	pool exhausted	无	
C: order timeout	4	下游节点	timeout	无	
D: 采样规则	0	无路径	无	10:00:00	

要求 写出 top-2、至少两条支持证据、一条反证和一个下一步验证动作。

练习参考答案：排序不是“唯一真相”

1. A 优先：最早出现、位于路径上游、存在相邻变更和指标语义；
2. B 次优：强症状证据，但缺少独立变更，可能是传播结果；
3. C 作为影响面证据，不宜直接当根因；
4. D 需要先检查观测链，因为它能解释“为什么看到异常”，不能解释真实业务损害；
5. 下一步：回放 cache 配置变更，核对 pre/post KPI 与同类实例。

若新 trace 证明 order 先于 cache，必须更新排序；好的 RCA 允许被证据推翻。

实验 2 最小实现骨架

```
past = data[data.ts < now]
base = rolling_median(past.metric, window="7d")
scale = rolling_mad(past.metric, window="7d") + 1e-6
current = samples_at(now)
score = 0.6745 * abs(current.metric - base) / scale

window = score > threshold
traces = select_traces(trace, scope=metric_scope, intervals=window)
candidates = rank_candidates(traces, logs, topology, changes)
evaluate(candidates, labels, metrics=["HR@1", "HR@3", "EXAM", "delay"])
```

- ▶ 聚合指标通常无 `trace_id`；先按对象与异常区间选 `Trace`，再关联 `Span`；
- ▶ 实验报告必须写清楚时间切分、缺失处理和阈值校准；
- ▶ 对每个候选保留 ‘supporting’ 与 ‘contradicting’ 字段。

可复现检查清单

- ▶ 数据版本与字段 schema;
- ▶ 训练/校准/测试的时间边界;
- ▶ 模型、窗口、阈值和随机种子;
- ▶ trace 采样率、ID 覆盖和时钟容忍度;
- ▶ 规则/模式库建立时间;
- ▶ 根因粒度与 top-k 定义;
- ▶ 缺失模态与负证据的处理;
- ▶ 失败案例、误报、延迟和运行开销。

提交物 代码、配置、候选表、评测表和一页失败分析; 没有失败分析的“高分”不完整。

分层对照算例：总错误率更高的版本可能并不更差

同一窗口比较旧版与新版；它们接收的短、长输入比例不同。表内为错误数 / 请求数。

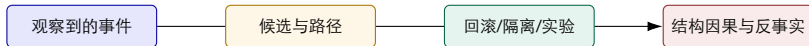
输入层	旧版	新版	层内错误率
短输入	9 / 900	1 / 100	两版均 1%
长输入	10 / 100	90 / 900	两版均 10%
总体	19 / 1,000	91 / 1,000	1.9% 对 9.1%
按 50:50 标准化	$0.5 \times 1\% + 0.5 \times 10\%$	同左	两版均 5.5%

先按输入长度对齐，再比较版本差异；必要时继续控制模型、租户和硬件。

推导与判断 该数据不支持“新版在同等输入下更易出错”。总体差异来自流量构成，但还不能排除未观测的混杂因素。

算例使用假设数据。

连接第 8 讲：从候选证据走向因果诊断



- ▶ 本讲：组织多源证据，产生可解释候选；
- ▶ 下讲：用 SCM、因果图和反事实区分相关与因果；
- ▶ 共同依据：高质量时间、拓扑、**trace**、日志和变更数据。

参考资料与延伸阅读

- ▶ Yu et al., *MicroRank: Latency Issue Localization with Extended Spectrum Analysis in Microservice*, WWW 2021, DOI 10.1145/3442381.3449905;
- ▶ Chen et al., *Nezha: Interpretable Fine-Grained Root Causes Analysis for Microservices on Multi-modal Observability Data*, ESEC/FSE 2023, DOI 10.1145/3611643.3616249;
- ▶ Yu et al., *ChangeRCA: Finding Root Causes from Software Changes in Large Online Systems*, FSE 2024, DOI 10.1145/3643728;
- ▶ *HotSpot: Anomaly Localization for Additive KPIs With Multi-Dimensional Attributes*, IEEE Access 2018, DOI 10.1109/ACCESS.2018.2804764;
- ▶ CauseLens, *Causality-based Interpretable Root Cause Analysis for Microservice Systems*, IWQoS 2025;
- ▶ Xu et al., *TELLER: Non-intrusive Cross-Layer Root-Cause Analysis for LLM Inference*, ASE 2026 (已录用), arXiv:2608.01975;
- ▶ 作者公开论文列表、脱敏案例与本地 ‘ai-infra-engineer-learning-main’ 工程教材。

DOI: [MicroRank](#); [Nezha](#); [ChangeRCA](#); [HotSpot](#)。

好的根因分析不
是给出一个名字
而是给出一条可复查、可
证伪、可行动的证据链

校企联合研究生课程 面向 AI 原生系统的智能运维